Thematic Assessment Report
Phase 1

TA REPORT PHASE 1

Persistent Memory Architecture in Agents Using DBMS

Abstract and Project Proposal

Submitted in partial fulfilment of course requirements
for Database Management Systems

Bachelor of Technology
Computer Science and Engineering
School of Engineering and Technology

Submitted to:
Dr. Poonam Sangwan
Faculty, DBMS
Department of CSE / SOET
Date of submission: _____

Submitted by:
Harshit Khemani (241302081)
Kush Ahuja (241302122)
Kushagra Agrawal (241302109)
Madhav Bassi (241302100)
B.Tech CSE (AI/ML), Section C

https://github.com/HKTITAN/dbms-agent-memory

SGT University
School of Engineering and Technology

Declaration. We declare that this proposal is our own work for DBMS TA-1.

Abstract

An AI agent that works with a team over many sessions has to remember what it learned: who owns a project, which database an application uses, who wrote a schema, and when any of that changed. Most agent memories keep this as free text in a file or as vectors in an index. Neither can declare a key, a foreign key, or a rule such as "a project has one current owner". This project treats agent memory as a database problem. We review how persistent memory for AI agents is built today, and we propose a relational memory store with two kinds of memory: episodic memory (what was said, in which session and turn) and semantic memory (facts stored as subject, predicate and object, with the time interval in which each fact held). The store has to support five operations on one schema with keys and checks: write, retrieve, update, forget, and consolidate. We plan to build it on SQLite with full-text search, so that it runs from a clone without a database server.

Keywords. agent memory · relational model · entity-relationship model · normalisation · integrity constraints · valid time · full-text search · SQLite

1. Introduction and problem statement

A language-model agent forgets everything when its context is cleared. Memory systems for agents store selected parts of past sessions outside the model and load them back when they are needed. Such a store is a database, but it is rarely designed as one. The same project can be called "Campus Navigator", "navigator" and "the app". A fact such as "the DBMS project team owns Campus Navigator" can stop being true. A claim can be a wrong guess that should be forgotten without losing the record of where it came from.

Problem. Design and implement a persistent memory for one AI agent in which these situations are handled by the database schema (keys, foreign keys, check constraints, and a validity interval) rather than by convention in application code.

2. Motivation

Each requirement of an agent memory has a standard database counterpart. Table 1 lists the ones this project depends on. Each row becomes a design decision in Phase 2.

Agent-memory need	DBMS concept
Keep what was learned after the session ends	Persistent storage in one database file
Recognise the same thing under different names	An entity table plus an alias table with a unique key
A fact changes over time (ownership moves)	Valid time: valid_from and valid_until on each fact
Know why the agent believes something	Provenance: a foreign key from a fact to the episode that supports it
Only one current owner of a project	A uniqueness constraint over current facts
Find memories by the words they contain	Full-text and keyword indexes
Forget a wrong memory but keep the audit trail	A status flag instead of a DELETE, protected by foreign keys
Summarise a session	A stored summary row: a materialised query result

Table 1: What an agent memory needs, and the DBMS concept that provides it.

3. Literature review

Du and colleagues describe memory for AI systems through six atomic operations: consolidation, indexing, updating, forgetting, retrieval, and compression [1]. MemGPT manages memory in tiers, like an operating system that pages data between the context window and external storage [2]. Mem0 extracts memories from a conversation and then decides whether to add, update, or delete stored entries [3]. Zep keeps memory as a temporal knowledge graph, so a relationship records when it was valid [4]. Lore, from Notion, keeps an agent's memory as pages in Notion databases, so that people can read and edit it [5].

These systems are mostly described in machine-learning terms: what to extract, how to rank, and when to summarise. The guarantees a DBMS gives are usually not stated: a key that prevents a duplicate alias, a foreign key that protects provenance, and a constraint that prevents two current owners. The relational model [6] and the entity-relationship model [7] give the vocabulary for stating them, and SQL:2011 standardised valid-time periods [8]. This project applies that vocabulary to agent memory. Phase 2 extends the survey and compares the substrates.

4. Objectives

1. Review persistent memory for AI agents and identify the memory operations a store must support.
2. Gather requirements and design a conceptual (ER) model with episodic and semantic memory.
3. Map the ER model to relational tables and normalise them, stating keys and functional dependencies.
4. Implement the schema in SQLite with primary keys, foreign keys, checks, and full-text search.
5. Implement write, retrieve (keyword and vector), update, forget, and consolidate in a small program.
6. Show one scripted agent session, with checks a reader can run again.

5. Scope and intended users

In scope	Out of scope
One agent, its sessions, and the turns it records (episodes)	Many concurrent writers and distributed storage
Entities with canonical names and aliases	A neural embedding model or a hosted vector database
Facts as subject, predicate, object with a validity interval	A language model that writes summaries
Keyword, full-text, and simple vector retrieval	Authentication and multi-user access control
Supersede, forget, consolidate, as-of queries	A graphical user interface

Table 2: Scope of the project.

Intended users. The direct user is an AI agent (a program that calls the store during a session). The people who use it are developers who build agents and need to inspect or correct what an agent remembers, and students of DBMS who want a small worked example of keys, constraints, normalisation, and temporal data.

6. Proposed tools

Software	Role
Python 3	Implementation language. `sqlite3` is in the standard library.
SQLite (with FTS5)	The database. One file, no server; foreign keys, checks, partial indexes, full-text search.

Software	Role
sqlite3 CLI	Running and inspecting SQL queries.
PostgreSQL	Considered in Phase 2 for comparison (range types and exclusion constraints).
Graphviz	Drawing the ER and architecture diagrams.
ReportLab	Printing the reports.
VS Code, Linux	Editor and operating system.

Table 3: Tools proposed for the project. Versions will be recorded when the program is run.

7. Preliminary entities

A first reading of the problem gives nine entity sets: agent, session, episode, entity, entity alias, fact, keyword, summary, and embedding. Figure 1 shows how they reference each other. Each arrow is a planned foreign key. Phase 2 turns this sketch into a full ER diagram with attributes, cardinalities, and constraints.

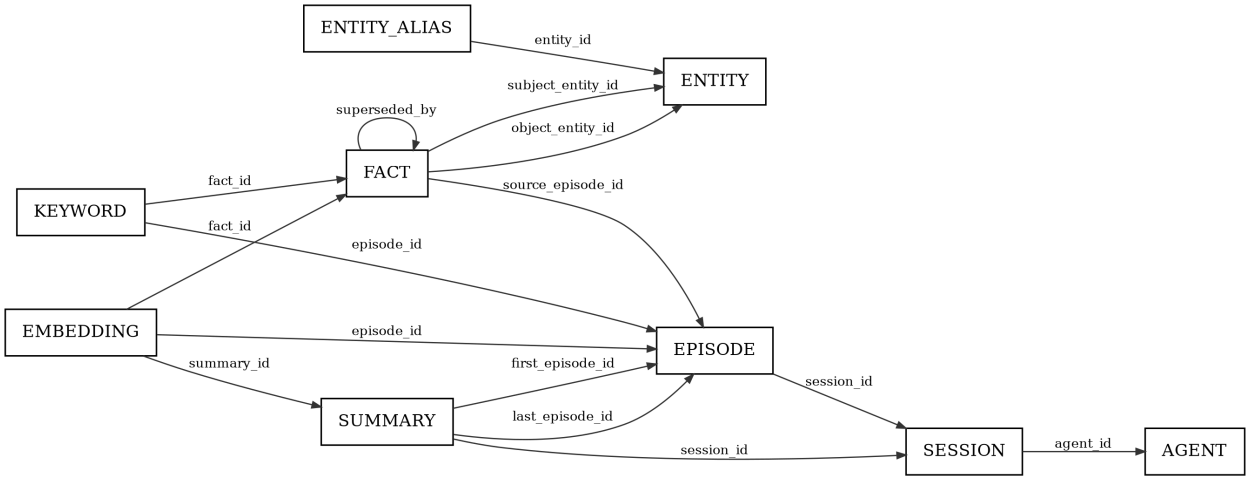


Figure 1: Preliminary entity sets. Each arrow points from the referencing table to the referenced table and is labelled with the planned foreign-key column.

8. Plan of work

Phase	Deliverable
Phase 1	This report: abstract, problem, motivation, literature review, objectives, scope, tools.
Phase 2	Survey of memory operations and substrates, requirements, ER diagram, relational schema, functional dependencies and normalisation, physical design plan.
Phase 3	Implementation in Python and SQLite, a scripted agent session, outputs, testing, and conclusions.

Table 4: The three phases of the project.

References

[1] Y. Du, W. Huang, D. Zheng, Z. Wang, S. Montella, M. Lapata, K.-F. Wong, and J. Z. Pan. *Rethinking Memory in AI: Taxonomy, Operations, Topics, and Future Directions*. arXiv:2505.00675, 2025.

[2] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez. *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560, 2023.

[3] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*. arXiv:2504.19413, 2025.

[4] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef. *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*. arXiv:2501.13956, 2025.

[5] Notion (makenotion). *lore: persistent, shared AI memory backed by Notion*. GitHub, MIT licence. https://github.com/makenotion/lore

[6] E. F. Codd. *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM 13(6):377-387, 1970.

[7] P. P.-S. Chen. *The Entity-Relationship Model: Toward a Unified View of Data*. ACM Transactions on Database Systems 1(1):9-36, 1976.

[8] K. Kulkarni and J.-E. Michels. *Temporal Features in SQL:2011*. ACM SIGMOD Record 41(3):34-43, 2012.